

Fehlermeldungen des Java Compilers (Temurin Java 25) verstehen



Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de, Raum 149B
Version: 1.0

1. Einleitung

Compiler und fehlermeldungen

- Fehlermeldungen des Compilers zu verstehen ist eine essentielle Fähigkeit und oft auch sehr einfach, da die Compiler sehr präzise geworden sind.
- In diesem Kapitel werden ausgewählte Fehlermeldungen des Java Compilers (Temurin Java 25) vorgestellt.
- Mit ein bisschen Übung werden Sie in der Lage sein die Fehlermeldungen schnell zu verstehen und die Fehler in Ihrem Code zu beheben. (Schneller als das Kopieren in den Chat Ihres AI-Assistenten.)

2. Fehlermeldungen des Java 25 Compilers

Fehlerhafte Programme und ihre Meldungen

2.1. Falscher Typ

```
1 int i = 0;
2
3 void main() {
4     byte b = i;
5     IO.println(b);
6 }
```

```
1 Bugs.java:4: error: incompatible types: possible lossy conversion from int to byte
2     byte b = i;
3           ^
```

2.2. Array Indizes

```
1 int[] a = new int[10];
2
3 void main(String[] args) {
4     long b = Integer.parseInt(args[0]);
5     IO.println(a[b]);
6 }
```

```
1 Bugs.java:5: error: incompatible types: possible lossy conversion from long to int
2     IO.println(a[b]);
3                   ^
```

2.3. Syntaxfehler

```
1 void printElement(int[] a, long b) throws IOException {
2     IO.println(a[(int) b]);
3     throw new IOException("Simulierte Ausnahme");
4 }
5
6 void main(String[] args) {
7     long b = Integer.parseInt(args[0]);
8     int[] a = { 10, 20, 30 };
9     printElement(a, b);
10 }
```

```
1 Bugs.java:3: error: not a statement
2     throw new IOException("Simulierte Ausnahme");
3     ^
4 Bugs.java:3: error: ';' expected
5     throw new IOException("Simulierte Ausnahme");
6           ^
```

2.4. Evolution von Java

```
1 void printElement(int[] a, long b) throws IOException {
2     IO.println(a[(int) b]);
3     throw new IOException("Simulierte Ausnahme");
4 }
```

```

5
6 void main(String[] args) throws IOException {
7     int[] a = { 10, 20, 30 };
8     printElement(a, Integer.parseInt(args[0]));
9 }

```

```

1 Bugs.java:1: error: class, interface, enum, or record expected
2 void printElement(int[] a, long b) throws IOException {
3 ^
4 Bugs.java:3: error: class, interface, enum, or record expected
5     throw new IOException("Simulierte Ausnahme");
6     ^
7 [... more errors ...]

```

2.5. static Kontext

```

1 public class Bugs {
2
3     void printElement(int[] a, long b) {
4         System.out.println(a[(int) b]);
5     }
6
7     public static void main(String[] args) {
8         int[] a = { 10, 20, 30 };
9         printElement(a, Integer.parseInt(args[0]));
10    }
11 }

```

```

1 Bugs.java:10: error: non-static method printElement(int[], long) cannot
2             be referenced from a static context
3     printElement(a, Integer.parseInt(args[0]));
4     ^

```

2.6. Definition von Klassen

```

1 public Class Util {
2     void printElement(int[] a, long b) {
3         System.out.println(a[(int) b]);
4     } }
5 public class Bugs {
6     public static void main(String[] args) {
7         printElement(new int[] { 10, 20, 30 }, Integer.parseInt(args[0]));
8     } }

```

```

1 Bugs.java:1: error: class, interface, annotation type, enum,
2             record, method or field expected
3 public Class Util {
4     ^
5 Bugs.java:3: error: class, interface, annotation type, enum,
6             record, method or field expected
7     } }
8     ^

```

2.7. No Symbol

```

1 public class Util {

```

▲ Achtung!

Die Definition von mehreren Klassen

```

2 static void printElement(int[] a, long
3     System.out.println(a[(int) b]);
4 }
5 }
6 public class Bugs {
7     public static void main(String[] args) {
8         int[] a = { 10, 20, 30 };
9         printElement(a, Integer.parseInt(args[0]));
10    }
11 }

```

in einer Datei ist nur in Java Skripten erlaubt (Java 19 und neuer).

```

1 Bugs.java:101
2 : error: cannot find symbol
3     printElement(a, Integer.parseInt(args[0]));
4     ^
5 symbol:   method printElement(int[],int)
6 location: class Bugs

```